DP Computer Science: B3 Object-Oriented Programming Linking Questions

Overview

Linking questions in the **'B3 OOP'** topic promote deeper understanding and connect concepts within computer science, fostering a holistic view of the field. They encourage students to see interconnectedness across different syllabus areas.

General Strategies for Using Linking Questions

Strategy	Description
Inquiry-Based Learning	Use questions to spark curiosity and encourage investigation
Class Discussions & Debates	Stimulate critical thinking and justification of reasoning
Activities & Mini-Projects	Design activities requiring application of knowledge from multiple areas
Concept Maps & Mind Maps	Create visual representations of connections between concepts
Formative Assessment	Use short quizzes or reflection journals to gauge understanding
Model Thinking	Explicitly model how to draw connections between different areas
Student-Generated Questions	Encourage students to formulate their own linking questions

Linking Question 1

> *"In what ways can OOP be applied to database development (A3)?"*

Connections Highlighted

This question links '**B3 Object-Oriented Programming**' with '**A3 Databases**'. It encourages students to think about how OOP concepts can be used to model and interact with database structures.

Key Concepts

Concept	Description
OOP to Database Mapping	Classes map to tables; objects map to rows; attributes map to columns
UML Class Diagrams	Used to design database schemas (ERDs)
Inheritance	Can model different types of records with common attributes
Composition/Aggregation	Represents complex relationships between entities
ORM (Object-Relational Mapping)	Technologies like SQLAlchemy, Hibernate

Teaching Activities

Activity	Description
Discussion Starter	How can real-world entities (customers, products, orders) be represented as objects and tables?
Design Activity	Design a database using UML class diagrams (e.g., library, online store)
Example	A `Book` class has attributes (title, author, ISBN) and methods (`checkout()`, `return()`). How does this map to a `Books` table?

Activity	Description
HL Extension	How can inheritance model different types of records? How can composition/aggregation represent relationships?

Enhanced Understanding

This question promotes understanding of how OOP principles translate to practical database design and how software applications interact with data storage systems.

Linking Question 2

> *"Is OOP necessary for all programming, or only in modelling complex situations (B2)?"*

Connections Highlighted

This question links **'B3 OOP'** with **'B2 Programming'** , encouraging students to compare OOP with other paradigms like procedural programming.

Key Concepts

Concept	Description
OOP vs. Procedural	Comparing paradigms: encapsulation, inheritance, polymorphism vs. functions and procedures
When to Use OOP	Complex systems, large-scale applications, GUI development
When Not to Use OOP	Simple scripts, performance-critical systems, small projects
Advantages of OOP	Maintainability, reusability, scalability
Disadvantages of OOP	Overhead, complexity, learning curve

Teaching Activities

Activity	Description
Class Debate	Debate advantages and disadvantages of OOP vs. procedural programming
Comparative Analysis	Solve the same problem using both paradigms (e.g., area of shapes, managing a list)
Case Studies	Compare simple command-line utilities vs. large GUI applications
Reflection	Reflect on personal programming experiences: when did OOP help? When was a simpler approach adequate?

Enhanced Understanding

This question deepens understanding of different programming paradigms and helps students make informed decisions about which approach is most suitable for different problems.

Linking Question 3

> "*How can design patterns in OOP facilitate the architecture of scalable and maintainable machine learning models (A4)?*"

Connections Highlighted

This question links **'B3 OOP'** with **'A4 Machine Learning'** . It encourages students to consider how OOP design patterns can improve ML code structure.

Key Concepts

Concept	Description
Design Patterns	Singleton, Factory, Observer, Strategy

Concept	Description
Scalability	How design patterns support growing systems
Maintainability	How design patterns improve code organisation
ML Applications	Factory for creating models; Observer for training notifications

Teaching Activities

Activity	Description
Introduction to Design Patterns	Introduce fundamental patterns and the problems they solve
ML Application	Factory pattern could create different ML models (classifiers, regressors); Observer pattern could notify components when training completes
Case Studies	Investigate popular ML libraries (scikit-learn, TensorFlow) for design patterns
Project Consideration	Challenge students to incorporate design patterns into their ML projects

Enhanced Understanding

This question encourages students to think innovatively about how different computer science domains can intersect and create well-structured, scalable systems.

Linking Question 4

> *"How can the principles of encapsulation and information hiding be applied to secure network communication (A2)?"*

Connections Highlighted

This question links **'B3 OOP'** with **'A2 Networks'** . It encourages students to consider how OOP principles apply to network security.

Key Concepts

Concept	Description
Encapsulation	Bundling data and methods; protecting internal state
Information Hiding	Restricting access to certain components
Network Protocols	TCP/IP encapsulates data into packets with headers and payloads
Encryption	Hiding data from unauthorised parties during transmission
Security Principles	Data integrity, confidentiality, authentication

Teaching Activities

Activity	Description
Explain Concepts	Define encapsulation and information hiding; emphasise protection of internal state
Analogy to Network Protocols	How does TCP/IP encapsulate data? How does encryption hide information?
Security Discussion	How does encapsulation prevent direct manipulation? How does encryption protect confidentiality?
Design Activity	Design a secure communication system, identifying where encapsulation and information hiding are crucial

Enhanced Understanding

This question highlights the practical application of OOP principles to real-world security challenges, demonstrating how software design principles translate to system security.

Summary: Linking Questions at a Glance

Question	Links	Key Concept
1. OOP in database development	B3 → A3	OOP to database mapping
2. OOP necessity for all programming	B3 → B2	Paradigm comparison
3. Design patterns in ML	B3 → A4	Patterns in ML architecture
4. Encapsulation in network security	B3 → A2	Security principles

```
text

┌──────────────────────────────────────────────────────────────┐
│                    KEY TAKEAWAYS                             │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│    ✅ Linking questions connect different syllabus areas     │
│    ✅ They promote deeper conceptual understanding           │
│    ✅ They encourage holistic thinking                       │
│    ✅ They can be used in various teaching strategies        │
│    ✅ They prepare students for assessment                   │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

> "*Linking questions encourage students to see the interconnectedness of concepts within computer science, fostering a holistic view of the field.*"